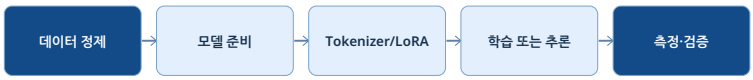
2026 보수교육 · 60분 실습 · 원본 교육과정 기준 · 적대적 검증 개선판 (2026-11-05)



01. 과정 개요

03차시 · Ollama와 Hugging Face 활용 1일차 13:00–14:00 / 60분 / Ubuntu · Python 3.11 · VS Code 2026학년도 보수교육(융합) 과정안내서 기준 / 작성일 2026-11-04

[교육 목표] 로컬 추론 서버와 Python 모델 로딩의 차이를 설명하고 두 방식으로 소형 LLM을 실행한다.

[시간 배분] 0–8분: Ollama와 Hugging Face 역할 이해 8–18분: 모델 형식과 추론 구조 분석 18–30분: Ollama 설치 및 명령 실습 30–45분: Transformers 모델 로딩 45–54분: 동일 질문 비교 및 API 호출 54–60분: 평가·오류 점검·정리

선택 모델: Qwen/Qwen2.5-0.5B-Instruct 및 qwen2.5:0.5b. 원본 과정안내서에서 특정 모델을 지정하지 않았으며 교육 편의상 선택했다.

02. 배경과 선수지식

LLM(대규모 언어 모델)은 가중치, 토크나이저, 추론 엔진, 입력 문장이 함께 작동해야 응답을 만든다.

Ollama(올라마, 로컬 모델 실행 도구): 모델 다운로드·관리·로컬 HTTP API 제공. Hugging Face(허깅 페이스, 모델과 데이터 공유 생태계): 모델 저장소, Transformers 등 라이브러리 제공.

[차이 비교] 실행: Ollama는 서버와 명령줄, Transformers는 Python 객체. 모델 파일: Ollama는 배포 패키지, Hugging Face는 모델 저장소 파일. 활용: Ollama는 로컬 서비스 배포, Transformers는 모델 내부 제어와 학습.

[선수지식] 01차시의 가상환경·pip 설치, 02차시의 토큰·Transformer·추론 개념을 활용한다.

주의: 같은 모델 계열이어도 양자화, 채팅 템플릿, 가중치 버전이 다르면 응답이 달라진다.

03. 핵심 원리

토큰(Token, 텍스트 처리 단위)은 문장을 모델이 처리하는 숫자 ID로 변환한 결과다.

Ollama 경로: 질문 → CLI/HTTP 요청 → Ollama 서버 → 모델 → 응답 JSON. Transformers 경로: 질문 → AutoTokenizer → AutoModelForCausalLM → generate() → decode().

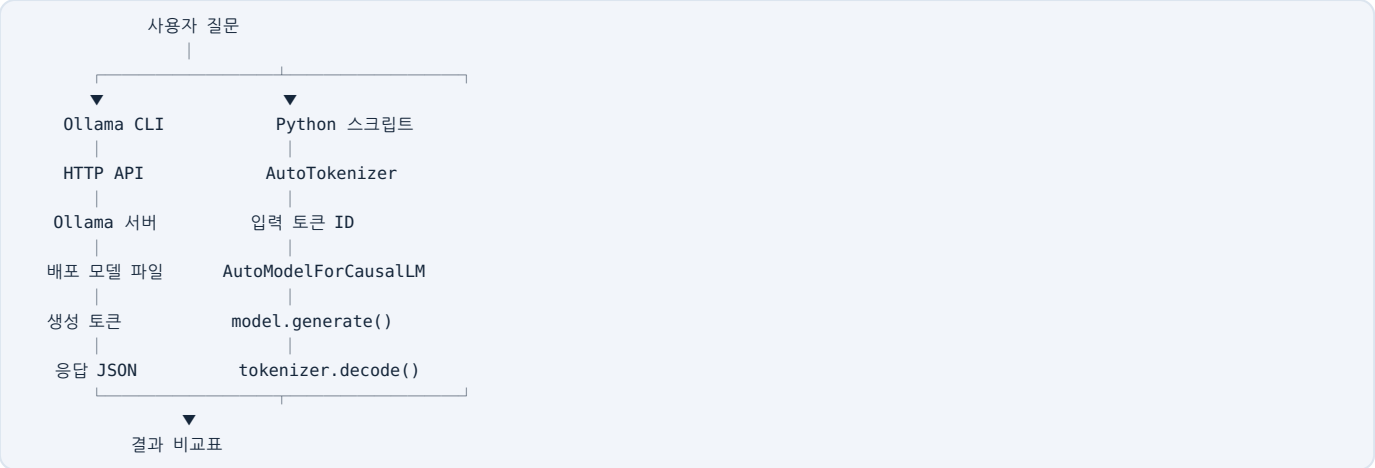
AutoTokenizer: 모델 전용 토크나이저를 불러온다. AutoModelForCausalLM: 다음 토큰을 생성하는 인과 언어 모델을 불러온다. apply_chat_template: 사용자 메시지를 모델 권장 대화 형식으로 변환한다. generate: 다음 토큰을 반복 생성한다.

CLI(씨엘아이, 명령줄 인터페이스)는 터미널 명령을 의미한다. HTTP API(에이치티티피이 에이피아이, 웹 요청 인터페이스)는 프로그램 간 요청·응답 규약이다.

양자화는 숫자 정밀도를 낮춰 메모리를 줄이는 기술이며 05차시에서 자세히 다룬다.

04. 구조도와 데이터 흐름

[전체 추론 구조]



검증 지점: Ollama는 ollama list, Python은 모델 ID·토큰 수·생성 문장을 기록한다.

중요: qwen2.5:0.5b와 Qwen/Qwen2.5-0.5B-Instruct는 동일한 체크포인트라고 단정할 수 없다.

05. 설치와 준비

[Ubuntu 명령어: 역할과 순서]

cd ~/lab_llm → 01차시 실습 폴더로 이동한다. 실제 생성된 경로를 사용한다. source .venv/bin/activate → Python 가상환경을 활성화한다. python --version → Python 3.11 환경인지 확인한다. python -m pip install -U transformers torch accelerate → 모델 로딩과 추론에 필요한 라이브러리를 설치한다.

curl -fsSL https://ollama.com/install.sh -o ollama-install.sh → 공식 설치 스크립트를 내려받는다. less ollama-install.sh → 외부 스크립트 내용을 검토한다. sh ollama-install.sh → 검토 후 설치한다. 교육장 관리 정책을 따른다. ollama --version → 설치 상태와 버전을 확인한다. ollama pull qwen2.5:0.5b → 실습용 모델을 내려받는다. ollama list → 다운로드한 모델 목록을 확인한다.

GPU 지원 PyTorch 빌드는 공식 설치 선택기를 이용해 장비별로 선택한다.

06. Ollama 명령 실습

[대화형 모델 실행] ollama run qwen2.5:0.5b 질문: UART가 무엇인지 두 문장으로 설명해줘.

[로컬 서버 모델 확인] curl http://127.0.0.1:11434/api/tags → 설치된 모델 목록을 JSON 형식으로 확인한다.

[Python API 실행] python lesson03_ollama_api.py → POST /api/generate 요청으로 답변을 받는다.

핵심 코드: payload = {"model": "qwen2.5:0.5b", "prompt": "UART가 무엇인지 두 문장으로 설명해줘.", "stream": False, "options": {"temperature": 0}}

stream=False: 조각별 스트리밍 대신 하나의 응답 JSON을 받는다. temperature=0: 임의성을 줄이는 설정이며 완전한 동일 출력을 보장하지 않는다.

기대 결과: model, answer, tokens가 표시된다. 실제 실행 여부는 별도로 기록한다.

07. Hugging Face 코드 해설

[Python 실습] python lesson03_hf_infer.py

핵심 코드: tokenizer = AutoTokenizer.from_pretrained(MODEL_ID) model = AutoModelForCausalLM.from_pretrained(MODEL_ID) model.eval() messages = [{"role": "user", "content": prompt}] inputs = tokenizer.apply_chat_template(messages, tokenize=True, add_generation_prompt=True, return_tensors="pt", return_dict=True) with torch.inference_mode(): outputs = model.generate(**inputs, max_new_tokens=80, do_sample=False, pad_token_id=tokenizer.eos_token_id)

[코드 풀이 — 이 코드가 무엇을 하는가] from_pretrained(): 가중치와 토크나이저를 다운로드·캐시한다. eval(): 모델을 평가 모드로 전환한다. inference_mode(): 역전파 추적 없이 추론한다. do_sample=False: 매 단계 확률 샘플링을 사용하지 않는다. max_new_tokens: 새로 생성할 최대 토큰 수를 제한한다.

기대 출력: model, input_tokens, output_tokens, answer. 숫자와 문장은 환경마다 달라질 수 있다.

08. 따라하기 비교 실습

[실습 절차]

1. ollama list로 모델 다운로드 상태를 확인한다.
2. python lesson03_ollama_api.py를 실행해 답변을 기록한다.
3. python lesson03_hf_infer.py를 실행해 답변을 기록한다.
4. 질문을 동일하게 유지하고 문장 수와 사실성을 비교한다.
5. 질문을 “PWM을 한 문장으로 정의해줘.”로 바꿔 반복한다.

[결과 기록 항목] Ollama 모델명: qwen2.5:0.5b Hugging Face 모델명: Qwen/Qwen2.5-0.5B-Instruct 질문: _____ Ollama 응답: _____ Hugging Face 응답: _____ 두 문장 조건 준수 여부: _____ 추론 시간(각각): _____ 오류 및 재시도 기록: _____

[해석 기준] 서로 다른 배포 파일·양자화·템플릿을 사용하므로 이 실습만으로 모델 성능 우열을 판단하지 않는다. 공정한 벤치마크는 같은 모델 파일·하드웨어·입력 조건이 필요하다.

09. 오류 점검 및 확장

[문제 해결] ollama: command not found → 설치 상태와 PATH 확인: command -v ollama 서버 연결 거부 → Ollama 서버 실행 상태 확인, 필요 시 ollama serve model not found → ollama pull qwen2.5:0.5b ModuleNotFoundError → which python, python -m pip show transformers 다운로드 실패 → 인터넷·방화벽·모델 접근 권한 확인 RAM/VRAM 부족 → 다른 프로그램 종료, 모델 규모·동시 실행 수 축소 응답이 다름 → 모델 버전·채팅 템플릿·양자화 설정 기록

[확장 실습]

1. 동일한 질문을 세 번 실행해 답변의 일관성을 비교한다.
2. max_new_tokens 값을 30과 80으로 바꿔 생성 길이를 비교한다.
3. Ollama 응답의 eval_count, total_duration 값을 확인한다. 시간 필드는 API 문서의 단위를 확인한 뒤 환산한다.

[보안 주의] 로컬 API를 인터넷에 무인증으로 노출하지 않는다. 실습에 개인정보나 기밀 자료를 입력하지 않는다. 모델 라이선스를 확인한다.

10. 평가 및 참고자료

[확인 문제]

1. Ollama와 Transformers의 역할 차이를 설명하라.
2. AutoTokenizer와 AutoModelForCausalLM의 기능을 구별하라.
3. 모델 응답이 달라지는 원인을 두 가지 이상 말하라.
4. 로컬 실행이 가능하려면 어떤 파일과 실행 환경이 필요한가?
5. 04차시 LoRA 학습에 적합한 실행 경로는 무엇인가?

[정답 요지]

1. 로컬 모델 서버 대 Python 모델 제어.
2. 토큰 변환 대 모델 추론.
3. 양자화·템플릿·가중치·버전 차이.
4. 모델 파일·토크나이저·추론 엔진·의존성.
5. Transformers와 PEFT 라이브러리.

[제출 체크리스트] ☐ Ollama 설치 목록 ☐ API 출력 ☐ Transformers 출력 ☐ 비교표 ☐ 오류 기록

[공식 참고자료 / 확인 기준일 2026-11-04] https://docs.ollama.com/linux https://docs.ollama.com/api/generate https://ollama.com/library/qwen2.5 https://huggingface.co/docs/transformers/quicktour https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct https://pytorch.org/get-started/locally/

실시간 공식 문서 변경 여부와 실제 추론 실행은 미검증. 다음 차시: 04 LoRA 원리와 PEFT.

적대적 검증·개선 기록 (2026-11-05)

반영: 실습 코드의 채팅 템플릿은 직접 토큰화하여 특수 토큰 중복 가능성을 줄였습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)